

EE673 Assignment 3

Mohd Amir, 220660

12th April, 2026

Contents

1	Wireshark Labs	2
1.1	IP Lab	2
1.2	ICMP Lab	5
2	Centralized and Distributed Routing Algorithms	7
2.1	Link-State Routing Algorithm	7
2.2	Distance-vector routing algorithm	10
3	HOL blocking	11
3.1	Transition Matrix and Stationary Distribution	11
3.2	Average Throughput of the Switch	12

1 Wireshark Labs

1.1 IP Lab

1. Source IP: 192.168.1.102
2. The value is **1**. This indicates that the payload is an **ICMP** (Internet Control Message Protocol) packet.

3. IP Header Length and Payload Size

- **IP Header Length:** The IP header length is determined from the Internet Header Length (IHL) field in the Wireshark capture. The IHL value specifies the number of 32-bit words in the header.

In this trace, $IHL = 5$. Therefore:

$$\text{Header Length} = 5 \times 4 = 20 \text{ bytes} \quad (1)$$

- **Payload Size:** The payload size is calculated by subtracting the header length from the Total Length field of the IP datagram.

From the captured packet:

- Total Length = 84 bytes
- Header Length = 20 bytes

$$\text{Payload Size} = 84 - 20 = 64 \text{ bytes} \quad (2)$$

- **Explanation:** The Total Length field includes both the IP header and the encapsulated data. By subtracting the header size (computed using IHL), we obtain the number of bytes in the payload. In this case, the payload corresponds to the ICMP message carried inside the IP datagram.

4. IP Datagram Fragmentation

- **Observation from Wireshark (Flags: 0x0):** The fragmentation status of the IP datagram is determined using the *Flags* field and the *Fragment Offset* field in the IP header.
 - The **More Fragments (MF)** bit is **0**.
 - The **Fragment Offset** is **0**.
- **Conclusion:** The IP datagram has **not been fragmented**.
- **Explanation:** A datagram is considered fragmented if either:
 - The MF (More Fragments) bit is set to 1, indicating more fragments follow, or
 - The Fragment Offset is non-zero, indicating this is a fragment of a larger datagram.

In this case, since both the MF bit is 0 and the Fragment Offset is 0, the datagram consists of a single, unfragmented packet.

5. Fields that Change Across IP Datagrams

- **Identification Field:** The Identification field changes for each IP datagram. It is used to uniquely identify fragments of a datagram during reassembly. Even when fragmentation is not used, this field is incremented for each new packet sent.

- **Time To Live (TTL):** The TTL field may change between datagrams depending on routing conditions. It represents the maximum number of hops a packet can traverse, and may decrease as the packet moves through routers.
- **Header Checksum:** The Header Checksum changes for every datagram because it is recomputed based on the contents of the IP header. Since fields like Identification and TTL may change, the checksum must also be updated.
- **Explanation:** Each ICMP message is encapsulated in a new IP datagram. Therefore, fields that are used for identification, routing control, and error detection are updated for each packet, while fields such as Source IP, Destination IP, and Protocol typically remain constant.

6. Constant Fields in IP Datagrams

- **Fields that Stay Constant (Observed):**

- Source IP Address
- Destination IP Address
- Protocol Field (ICMP = 1)
- Header Length (IHL)

These fields remain constant because all ICMP messages are sent between the same source and destination using the same protocol.

- **Explanation:** Constant fields define the communication endpoints and protocol, while changing fields ensure correct packet identification, routing control, and error detection for each individual datagram.

7. Pattern in the Identification Field

- **Observed Pattern:** The values in the Identification field increase sequentially for each successive IP datagram (increasing time). The Identification field follows a monotonically increasing sequence, typically incrementing by 1 for each successive packet.

8. Identification Field and TTL Value in ICMP TTL-Exceeded Replies

- **Identification Field:** The value observed in the Identification field is **0x9dc (40136)**.
- **TTL Field:** The value observed in the TTL field is **255**.
- **Observation Method:** These values were obtained by inspecting the IP header of the ICMP TTL-exceeded packets sent by the first-hop router in the Wireshark capture.

9. Consistency of Identification and TTL Values

- **Observation:** Upon examining multiple ICMP TTL-exceeded replies from the first-hop router:
 - The **TTL values remain constant**.
 - The **Identification field changes across packets**.
- **Explanation:** These ICMP TTL-exceeded messages are newly generated packets created by the router, rather than forwarded packets. Therefore:
 - The **TTL field** is initialized to a fixed value for each packet and is not affected by prior transmissions.
 - The **Identification field** is assigned independently by the router's IP stack and may either remain constant or change depending on implementation.

- **Conclusion:** The behavior of these fields depends on how the router generates packets, but the TTL is typically constant due to fresh initialization, while the Identification field reflects router-specific assignment behavior.

Note: Although the Identification field is primarily used for fragmentation and re-assembly, in this case the ICMP TTL-exceeded packets are not fragmented. Therefore, the observed behavior of the Identification field is due to the router's packet generation mechanism rather than fragmentation.

10. Fragmentation of ICMP Echo Request (Packet Size = 2000)

- The ICMP Echo Request message has **been fragmented across multiple IP datagrams**, as indicated by the presence of 2 fragments with the same Identification value and appropriate MF flag and Fragment Offset values.

11. Analysis of the First Fragment of a Fragmented IP Datagram

- **Indication of Fragmentation:** The IP datagram is fragmented as indicated by:
 - The **More Fragments (MF)** flag being set to **1**
 - The presence of non-zero **Fragment Offset** values in second fragment.
- **Identification of the First Fragment:** The first fragment is identified by:
 - **Fragment Offset = 0**
 - **More Fragments (MF) flag = 1**
- **Distinguishing First and Later Fragments:**
 - The **first fragment** has Fragment Offset = 0.
 - **Later fragment** have Fragment Offset > 0.
- **Total Length of the first IP Datagram: 1500 bytes.**
- **Explanation:** The Total Length represents the size of this individual fragment, including header and payload. Since fragmentation occurs due to MTU limitations, each fragment (except the last) typically has a size close to the MTU.

12. Analysis of the Second Fragment of a Fragmented IP Datagram

- **Identification of the Second Fragment:** The second fragment is identified by having the same **Identification field** as the first fragment and a **Fragment Offset** greater than zero.
- **Indication that this is not the First Fragment:** The **Fragment Offset** is **1480**, which is greater than zero. This indicates that the fragment does not start at the beginning of the original datagram and is therefore not the first fragment.
- **Are there More Fragments?**
 - since MF = 0, therefore this is the last fragment.
- **Explanation:** The Fragment Offset field specifies the position of this fragment within the original datagram. A non-zero value indicates it is not the first fragment. The MF flag indicates whether additional fragments are expected.

13. Fields that Change Between First and Second Fragment

- **Fragment Offset:** The Fragment Offset changes from **0** in the first fragment to **1480** in the second fragment.
- **More Fragments (MF) Flag:** The MF flag is **1** in the first fragment and **0** in the second fragment.

- **Total Length:** The Total Length field differs between fragments, as each fragment carries a different portion of the original data.
- **Header Checksum:** The Header Checksum changes because it is recomputed for each fragment based on the modified header fields.

14. Number of Fragments Created from original datagram: 3

15. Fields that Change Among IP Fragments

- **Fragment Offset**
- **More Fragments (MF) Flag**
- **Total Length**
- **Header Checksum**

1.2 ICMP Lab

1. Host and Destination IP Addresses

- IP Address of the Host (Source): **192.168.1.101**
- IP Address of the Destination: **143.89.14.34**

2. Absence of Port Numbers in ICMP Packets

- **Observation:** ICMP packets do not contain source and destination port numbers in their headers.
- **Explanation:** Port numbers are a feature of the Transport Layer (specifically protocols like TCP and UDP). They are used to multiplex and demultiplex data, directing it to specific applications or processes running on a destination host (e.g., routing web traffic to port 80).

ICMP, on the other hand, is a Network Layer protocol. Its primary function is host-to-host communication for network-level diagnostics, control, and error reporting. Because ICMP messages are intended for the network software (the IP stack) of the host operating system itself, rather than a specific user-space application, there is no need for application-level port numbers.

3. ICMP Echo Request Analysis

- **ICMP Type and Code:**
 - **Type:** 8 (Echo (ping) request)
 - **Code:** 0
- **Other Fields in the ICMP Packet:** Besides the Type and Code, the ICMP header for an Echo Request contains the following fields:
 - **Checksum:** Used for error-checking the ICMP header and data.
 - **Identifier (BE & LE):** Used to match requests with replies.
 - **Sequence Number (BE & LE):** Used to track individual packets within a ping session.
 - **Data:** The payload, typically containing a timestamp and padded bytes.
- **Size of Specific Fields:**
 - **Checksum:** 2 bytes (16 bits)
 - **Identifier:** 2 bytes (16 bits)
 - **Sequence Number:** 2 bytes (16 bits)

4. ICMP Echo Reply Analysis

- **ICMP Type and Code:**
 - **Type:** 0 (Echo (ping) reply)
 - **Code:** 0
- **Other Fields in the ICMP Packet:** The Echo Reply header contains the same structural fields as the Echo Request:
 - **Checksum:** Used for error-checking the ICMP header and data.
 - **Identifier (BE & LE):** Copied directly from the request to identify the session.
 - **Sequence Number (BE & LE):** Copied directly from the request to match this specific reply to the corresponding request.
 - **Data:** The payload, mirroring the data originally sent in the request.
- **Size of Specific Fields:**
 - **Checksum:** 2 bytes (16 bits)
 - **Identifier:** 2 bytes (16 bits)
 - **Sequence Number:** 2 bytes (16 bits)

5. Host and Target Destination IP Addresses

- IP Address of the Host (Source): **192.168.1.101**
- IP Address of the Destination: **138.96.146.2**

6. IP Protocol Number for UDP Probe Packets

- No, the IP protocol number would not be 01. It would be **17**.
- The IP Protocol field identifies the upper-layer protocol encapsulated within the IP datagram payload. When Windows runs a traceroute, it sends ICMP Echo Request packets, so the Protocol field is set to 1 (for ICMP). However, Unix/Linux traceroute implementations typically use UDP datagrams for their probe packets. Because the payload is now a UDP segment, the IP Protocol field must be set to 17, which is the standard protocol number for UDP.

7. Comparison of Traceroute ICMP vs. Ping ICMP Packets

- Yes, while the ICMP payload itself is structurally similar (both are Type 8, Code 0 Echo Requests), there is a crucial difference in the encapsulating **Internet Protocol (IP) header**, specifically regarding the **Time to Live (TTL)** field.
- **Explanation:**
 - In a standard **ping query**, the IP TTL is set to a relatively high default value (e.g., 64, 128, or 255) to ensure the packet has enough "life" to traverse all necessary routers and reach the target destination.
 - In a **traceroute query**, the packets are sent with deliberately manipulated IP TTL values. The first packet is sent with a TTL of 1, the second with a TTL of 2, and so forth. This intentional throttling causes intermediate routers to discard the packet and return an ICMP "TTL Exceeded" message, which is the exact mechanism traceroute uses to map the network path.

8. Contents of the ICMP Error Packet

- The ICMP error packet (such as a "Time-to-live exceeded" message) contains a larger payload with additional fields compared to a standard ICMP Echo Request/Reply.

- **Included Fields (Payload):** The data portion of the ICMP error message includes:
 - (a) The complete **Internet Protocol (IP) header** of the original, discarded packet.
 - (b) The **first 8 bytes** of the original packet's data payload (which typically contains the UDP header or the original ICMP Echo header).
- **Explanation:** When a router discards a packet, it must inform the sending host. However, the sending host needs context to understand which of its many outgoing connections failed. By embedding a copy of the original IP header and the first 8 bytes of the original payload, the router provides enough information (such as original port numbers or sequence identifiers) for the host's operating system to precisely match the error to the specific application or process that generated the dropped packet.

9. Analysis of the Final Three ICMP Packets

- **Observation:** Depending on the operating system's implementation of traceroute, the last three packets differ from the intermediate ICMP "Time-to-live exceeded" (Type 11) error packets:
 - **Windows (tracert):** The last three packets are **ICMP Echo Replies** (Type 0, Code 0).
- **Why they are different:** These packets are different because the original probe packets **finally reached the target destination host** before their TTL value dropped to zero. Therefore, no TTL Exceeded error was triggered.
- **Explanation:**
 - **For ICMP probes (Windows):** The destination host receives an Echo Request and naturally processes it by sending back a standard Echo Reply, indicating a successful arrival.

In both implementations, receiving these specific responses signals to the traceroute utility that the final destination has been reached, prompting the program to terminate.

10. Analysis of Link Delays and Router Locations

- **Significant Delay Observation:** Yes, upon examining the traceroute measurements in Figure 4, there is a link with a significantly longer delay compared to the others. The substantial jump in Round Trip Time (RTT) occurs between **Hop 9** (average delay: 24 ms) and **Hop 10** (average delay: 97 ms).
- **Router Names and Geographical Locations:** Based on the naming conventions of the routers at the ends of this high-delay link:
 - **Router 1 (Hop 9):** att-gw.nyc.opentransit.net
 - **Router 2 (Hop 10):** P4-0.PASCR1.Pastourelle.opentransit.net
- **Conclusion:** By decoding the geographical abbreviations within these router names, it can be inferred that Router 1 is located in **New York City, USA** (indicated by "nyc"), and Router 2 is located in **France** (indicated by "Pastourelle" and the subsequent French hops like Aubervilliers). The sudden increase in delay of approximately 74 ms is physically consistent with the propagation delay required for packets to traverse a transatlantic fiber-optic link between North America and Europe.

2 Centralized and Distributed Routing Algorithms

2.1 Link-State Routing Algorithm

*Note: The results can be verified using my **solution2.1.py** which contains the detailed implementation of the above algorithm.*

In the Link-State (LS) routing algorithm, every node possesses complete global knowledge of the network topology and link costs. Each node independently runs Dijkstra's algorithm to compute the shortest path to all other nodes.

Notations and Definitions

Before detailing the step-by-step execution, we define the standard variables used in Dijkstra's algorithm:

- N' : The set of nodes whose shortest path from the source node is definitively known and finalized.
- $D(v)$: The cost of the current lowest-cost path from the source node to destination node v . As the algorithm progresses, this value is updated if a shorter path is discovered.
- $p(v)$: The previous node (predecessor) along the current shortest path from the source to node v . This is crucial for constructing the final routing table (the forwarding table).
- $c(u, v)$: The direct link cost between node u and node v . If no direct physical link exists between the two nodes, $c(u, v) = \infty$.

For the given graph, the node set is $\{0, 1, 2, 3\}$ and the bidirectional link costs are: $c(0, 1) = 1$, $c(0, 2) = 3$, $c(0, 3) = 7$, $c(1, 2) = 1$, and $c(2, 3) = 2$.

Below are the step-by-step execution tables and the final routing tables computed at each node.

1. Execution at Node 0

Dijkstra's Algorithm Table:

Step	N' (Known Nodes)	$D(1), p(1)$	$D(2), p(2)$	$D(3), p(3)$
0	$\{0\}$	1, 0	3, 0	7, 0
1	$\{0, 1\}$		2, 1	7, 0
2	$\{0, 1, 2\}$			4, 2
3	$\{0, 1, 2, 3\}$			

Routing Table for Node 0:

Destination	Next Hop	Total Cost
1	1	1
2	1	2
3	1	4

2. Execution at Node 1

Dijkstra's Algorithm Table:

Note: In Step 1, nodes 0 and 2 tie for the minimum cost. Node 0 is selected arbitrarily, but selecting Node 2 yields the exact same final routing table.

Step	N' (Known Nodes)	$D(0), p(0)$	$D(2), p(2)$	$D(3), p(3)$
0	{1}	1, 1	1, 1	∞
1	{1, 0}		1, 1	8, 0
2	{1, 0, 2}			3, 2
3	{1, 0, 2, 3}			

Routing Table for Node 1:

Destination	Next Hop	Total Cost
0	0	1
2	2	1
3	2	3

3. Execution at Node 2

Dijkstra's Algorithm Table:

Step	N' (Known Nodes)	$D(0), p(0)$	$D(1), p(1)$	$D(3), p(3)$
0	{2}	3, 2	1, 2	2, 2
1	{2, 1}	2, 1		2, 2
2	{2, 1, 0}			2, 2
3	{2, 1, 0, 3}			

Routing Table for Node 2:

Destination	Next Hop	Total Cost
0	1	2
1	1	1
3	3	2

4. Execution at Node 3

Dijkstra's Algorithm Table:

Step	N' (Known Nodes)	$D(0), p(0)$	$D(1), p(1)$	$D(2), p(2)$
0	{3}	7, 3	∞	2, 3
1	{3, 2}	5, 2	3, 2	
2	{3, 2, 1}	4, 1		
3	{3, 2, 1, 0}			

Routing Table for Node 3:

Destination	Next Hop	Total Cost
0	2	4
1	2	3
2	2	2

2.2 Distance-vector routing algorithm

*Note: The results can be verified using my **solution2.2.py** which contains the detailed implementation of the above procedural algorithm.*

Unlike the Link-State algorithm, the Distance-Vector (DV) routing algorithm is decentralized and distributed. No single node possesses complete topological knowledge. Instead, nodes compute paths iteratively using the Bellman-Ford equation:

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\} \quad (3)$$

where $D_x(y)$ is the minimum cost to reach destination y from node x , and v iterates over all immediate neighbors of x .

1. Implementation Routines

To emulate this distributed behavior, two primary routines are defined for every node (0 through 3):

- **rtinitX():** This routine is called once at the start of the emulation. Node X initializes its Distance Table using only the costs of its directly attached physical links. It then packages this initial vector of minimum costs into a routing packet (**rtpkt**) and transmits it to all immediate neighbors via the **tolayer2()** network emulation function.
- **rtupdateX(rtpkt):** This routine is triggered whenever node X receives a routing packet from a neighbor. The node updates its internal Distance Table with the neighbor's new minimum costs. It then recalculates its own shortest paths using the Bellman-Ford equation. Crucially, a new routing packet is generated and sent to its neighbors *only if* this recalculation results in a strictly lower minimum cost to any destination. This condition ensures that the network eventually converges and stops transmitting updates.

2. Convergence and Final Cost Vectors

Through the asynchronous exchange of **rtpkt** packets, the network iteratively discovers the global shortest paths. The final, converged minimum cost vectors for each node mirror the total costs found via Dijkstra's algorithm, confirming the accuracy of the distributed computation.

Node	Cost to 0	Cost to 1	Cost to 2	Cost to 3
0	0	1	2	4
1	1	0	1	3
2	2	1	0	2
3	4	3	2	0

Table 1: Final Converged Minimum Cost Vectors (D_x)

3 HOL blocking

Note: The theoretical results and stationary distribution derived below can be verified by the code `solution3.py`.

3.1 Transition Matrix and Stationary Distribution

Let the state space of the Markov chain be $S = \{11, 12, 21, 22\}$. Based on the collision and independent uniform arrival probabilities, the transition matrix P is defined as follows:

- **Transitions from Conflict States (eg. State 11):** Both packets request output port 1. The switch randomly selects one to transfer.
 - With probability 0.5, Queue 1 is served. A new packet arrives at Queue 1 with a uniformly random destination (0.5 for port 1; 0.5 for port 2). Queue 2 retains its packet destined for port 1. This leads to State 11 (prob: $0.5 \times 0.5 = 0.25$) or State 21 (prob: $0.5 \times 0.5 = 0.25$).
 - With probability 0.5, Queue 2 is served. A new packet arrives at Queue 2 (0.5 for port 1; 0.5 for port 2). Queue 1 retains its packet destined for port 1. This leads to State 11 (prob: $0.5 \times 0.5 = 0.25$) or State 12 (prob: $0.5 \times 0.5 = 0.25$).

Summing these probabilities for an initial State 11, the next state probabilities are: $P(11 \rightarrow 11) = 0.5$, $P(11 \rightarrow 12) = 0.25$, $P(11 \rightarrow 21) = 0.25$, and $P(11 \rightarrow 22) = 0$. Symmetric logic applies to State 22.

- **Transitions from Non-Conflict States (e.g., State 12):** There is no conflict. Both HOL packets are successfully transferred. Two new packets advance to the head of the queues. Since destinations are assigned uniformly and independently across packets, the next state can be any of the four states with equal probability: $0.5 \times 0.5 = 0.25$. The same logic applies to State 21.

$$P = \begin{bmatrix} 0.5 & 0.25 & 0.25 & 0 \\ 0.25 & 0.25 & 0.25 & 0.25 \\ 0.25 & 0.25 & 0.25 & 0.25 \\ 0 & 0.25 & 0.25 & 0.5 \end{bmatrix} \quad (4)$$

The transition matrix for this Markov chain is as follows:

	State 11	State 12	State 21	State 22
State 11	0.5	0.25	0.25	0
State 12	0.25	0.25	0.25	0.25
State 21	0.25	0.25	0.25	0.25
State 22	0	0.25	0.25	0.5

Let $\pi = [\pi_{11}, \pi_{12}, \pi_{21}, \pi_{22}]$ be the stationary distribution. Mathematically, π is the left eigenvector of the transition matrix P corresponding to the eigenvalue $\lambda = 1$, satisfying the equation $\pi P = 1 \cdot \pi$ (subject to the normalization constraint $\sum \pi_i = 1$).

Solving this corresponding system of linear equations:

$$\begin{aligned}
\pi_{11} &= 0.5\pi_{11} + 0.25\pi_{12} + 0.25\pi_{21} \\
\pi_{12} &= 0.25\pi_{11} + 0.25\pi_{12} + 0.25\pi_{21} + 0.25\pi_{22} \\
\pi_{21} &= 0.25\pi_{11} + 0.25\pi_{12} + 0.25\pi_{21} + 0.25\pi_{22} \\
\pi_{22} &= 0.25\pi_{12} + 0.25\pi_{21} + 0.5\pi_{22}
\end{aligned}$$

By symmetry, $\pi_{12} = \pi_{21}$. Substituting this into the first equation yields $0.5\pi_{11} = 0.5\pi_{12}$, meaning $\pi_{11} = \pi_{12}$. Applying the same to the fourth equation yields $\pi_{22} = \pi_{12}$.

Therefore, all states are equally likely. Applying the normalization condition ($\sum \pi_i = 1$):

$$\pi = [0.25, 0.25, 0.25, 0.25]$$

3.2 Average Throughput of the Switch

The throughput of the switch in a given time slot depends on the state of the Markov chain:

- In states **12** and **21** (total probability = $0.25 + 0.25 = 0.5$), there is no conflict, and **2 packets** are successfully transferred.
- In states **11** and **22** (total probability = $0.25 + 0.25 = 0.5$), there is a conflict, and only **1 packet** is successfully transferred.

The expected absolute average throughput $E[T]$ is calculated as:

$$E[T] = (0.5 \times 2 \text{ packets}) + (0.5 \times 1 \text{ packet}) = 1.5 \text{ packets per slot}$$

To find the **normalized throughput** (efficiency), we divide the expected throughput by the maximum possible capacity of the 2×2 switch (which is 2 packets per slot):

$$\text{Normalized Throughput} = \frac{1.5}{2} = 0.75$$

Conclusion: A perfect 2×2 switch without HOL blocking would process 2 packets per slot. Due to HOL blocking, the absolute average throughput is reduced to 1.5 packets per slot. This means the normalized throughput is **0.75**, indicating the switch operates at only **75%** capacity.